

PROGRAMAÇÃO EM HASKELL



Capítulo 2 - Primeiros Passos

Glasgow Haskell Compiler (GHC)

- GHC é a implementação líder de Haskell, compreendendo **compilador** e **interpretador**
- A natureza interativa do interpretador o torna adequado para o **ensino** e a **prototipagem**
- GHC está disponível gratuitamente em
www.haskell.org/downloads
- Outros recursos disponíveis em
www.haskell.org

Iniciando o GHCi

O interpretador pode ser iniciado a partir do prompt de comando do terminal \$ digitando **ghci**

```
$ ghci
```

```
GHCi, version X: http://www.haskell.org/ghc/ :? for help
```

```
Prelude>
```

O prompt do GHCi > significa que o intérprete agora está pronto para avaliar uma expressão

Por exemplo, ele pode ser usado como uma calculadora de desktop para avaliar expressões numéricas simples

```
> 2+3*4
```

```
14
```

```
> (2+3)*4
```

```
20
```

```
> sqrt (3^2 + 4^2)
```

```
5.0
```

O padrão Prelude

Biblioteca padrão de Haskell com grande número de funções. Além das funções numéricas familiares, como $+$ e $*$, a biblioteca também fornece muitas funções úteis em **listas**

- Selecione o primeiro elemento de uma lista

```
> head [1,2,3,4,5]  
1
```

- Remova o primeiro elemento de uma lista

```
> tail [1,2,3,4,5]  
[2,3,4,5]
```

- Selecionar o enésimo elemento de uma lista

```
> [1,2,3,4,5] !! 2  
3
```

- Selecionar os primeiros n elementos numa lista

```
> take 3 [1,2,3,4,5]  
[1,2,3]
```

- Remova os primeiros n elementos de uma lista

```
> drop 3 [1,2,3,4,5]  
[4,5]
```

- Calcular o tamanho de uma lista

```
> length [1,2,3,4,5]  
5
```

- Calcular a soma de uma lista de números

```
> sum [1,2,3,4,5]  
15
```

- Calcule o produto de uma lista de números

```
> product [1,2,3,4,5]  
120
```

- Anexe duas listas

```
> [1,2,3] ++ [4,5]  
[1,2,3,4,5]
```

- Reverta uma lista

```
> reverse [1,2,3,4,5]  
[5,4,3,2,1]
```

Aplicação de função

Na **matemática**, a aplicação de função é denotada por parênteses, e a multiplicação é frequentemente denotada usando justaposição ou espaço

$$f(a,b) + c \, d$$

Aplique a função **f** para **a** e **b**, e adicione o resultado ao produto de **c** e **d**

Em **Haskell**, a aplicação da função é denotada usando o espaço, e a multiplicação é denotada usando `*`

```
f a b + c*d
```

Como anteriormente, mas na
sintaxe Haskell

Além disso, assume-se que a aplicação de função tem **prioridade mais elevada** do que todos os outros operadores

$f \ a \ + \ b$

Significa $(f \ a) \ + \ b$, em vez de $f \ (a \ + \ b)$

Exemplos

Matemática

$f(x)$

$f(x, y)$

$f(g(x))$

$f(x, g(y))$

$f(x)g(y)$

Haskell

$f\ x$

$f\ x\ y$

$f\ (g\ x)$

$f\ x\ (g\ y)$

$f\ x\ * \ g\ y$

Haskell Scripts

- Assim como as funções na biblioteca padrão, você pode definir suas próprias funções
- Novas funções são definidas em um **script**, um arquivo de texto com uma sequência de definições
- Por convenção, scripts Haskell têm um sufixo **.hs** em seu nome de arquivo. Isso não é obrigatório, mas é útil para fins de identificação

Meu Primeiro Script

Ao desenvolver um script Haskell, é útil manter duas janelas abertas, uma executando um editor para o script e a outra executando o GHCi

Inicie um editor, digite as duas definições de função a seguir e salve o script como **test.hs**

```
double x = x + x
```

```
quadruple x = double (double x)
```

Deixando o editor aberto, em outra janela inicie o GHCi com o novo script

```
$ ghci test.hs
```

Agora, a biblioteca padrão e o arquivo **test.hs** estão carregados, e as funções de ambos podem ser usadas

```
> quadruple 10  
40
```

```
> take (double 2) [1,2,3,4,5,6]  
[1,2,3,4]
```

Deixando o GHCi aberto, retorne ao editor, adicione as duas definições a seguir e salve novamente

```
factorial n = product [1..n]  
average ns = sum ns `div` length ns
```

Observe

- `div` é colocado entre aspas para trás, não para frente
- `x `f` y` é apenas **açúcar sintático** para `f x y`

O GHCi não detecta automaticamente a alteração do script, portanto, deve-se executar o comando **reload** antes que as novas definições possam ser usadas

```
> :reload
Reading file "test.hs"

> factorial 10
3628800

> average [1,2,3,4,5]
3
```

Comandos GHCi úteis

<u>Comando</u>	<u>Significado</u>
:load <i>name</i>	carrega script <i>name</i>
:reload	recarrega script atual
:set editor <i>name</i>	define editor para <i>name</i>
:edit <i>name</i>	edita script <i>name</i>
:edit	edita script atual
:type <i>expr</i>	mostra o tipo de <i>expr</i>
:?	Mostra todos os comandos
:quit	sai do GHCi

Requisitos de nomeação

- Nomes de função e argumento devem começar com letra minúscula, por exemplo

myFun

fun1

arg_2

x'

- Por convenção, argumentos de lista têm um sufixo **s** em seu nome, por exemplo

xs

ns

nss

Regra de layout

Numa sequência de definições, cada definição deve começar precisamente na mesma coluna

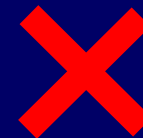
```
a = 10
b = 20
c = 30
```



```
a = 10
  b = 20
c = 30
```



```
a = 10
b = 20
  c = 30
```



A regra de layout evita o uso de sintaxe explícita para indicar o agrupamento de definições

```
a = b + c
  where
    b = 1
    c = 2
d = a * 2
```

agrupamento
implícito

significa

```
a = b + c
  where
    {b = 1;
     c = 2}
d = a * 2
```

agrupamento
explícito

Exercícios

- (1) Experimente os slides 2-7 e 13-16 usando GHCi
- (2) Corrija os erros de sintaxe no programa abaixo e teste sua solução usando o GHCi

```
N = a 'div' length xs
  where
    a = 10
    xs = [1,2,3,4,5]
```

- (3) Mostre como a função de biblioteca **last**, que seleciona o último elemento numa lista, pode ser definida usando funções introduzidas nesta aula. Dá para pensar em outra definição? Mostre!
- (4) Coloque entre parêntesis as seguintes expressões numéricas

$$2^3 * 4$$

$$2 * 3 + 4 * 5$$

$$2 + 3 * 4^5$$

- (5) Da mesma forma, mostre como a função de biblioteca **init**, que remove o último elemento de uma lista, pode ser definida de duas maneiras diferentes